

DL

Understood. We'll go step by step, in simple SPPU-style explanation, without unnecessary extras.

Assignment 1 – Step 1: Basic Understanding (Very Simple)

Aim (in simple words)

To predict house prices using:

- Linear Regression
 - Deep Neural Network on the Boston Housing dataset.
-

1. What is Linear Regression?

Linear regression is a method used to find the relationship between:

- Input variables (like rooms, crime rate)
- Output variable (house price)

It assumes the relationship is **linear (straight line)**.

Simple idea:

If number of rooms increases → price increases

If crime rate increases → price decreases

So model learns:

Price = combination of all input features

2. What is Deep Neural Network (DNN)?

A Deep Neural Network is a machine learning model inspired by the human brain.

It has:

- Input layer (takes data)

- Hidden layers (process data)
- Output layer (gives result)

Each layer contains neurons which learn patterns.

In this experiment:

- Input = 13 features of house
 - Output = predicted price
-

3. Why combine Linear Regression with DNN?

Normal linear regression:

- Works well for simple relationships

Deep Neural Network:

- Can learn complex patterns
- Gives better prediction accuracy

So we use DNN for regression task.

4. What is Boston Housing Dataset?

It is a dataset used for learning regression.

It contains:

- 506 house samples
- 13 features like:
 - Crime rate
 - Number of rooms
 - Tax rate
 - Distance to workplace

Output:

- MEDV = house price

Boston Housing Dataset

5. Overall Flow of Experiment

You should remember this flow (very important for viva):

1. Load dataset
2. Preprocess data (standardization)
3. Split into train and test
4. Build neural network
5. Train model
6. Evaluate model
7. Predict output

Q1. What is Linear Regression?

Ans:

Linear Regression is a statistical and machine learning technique used to find the relationship between a dependent variable (output) and one or more independent variables (inputs). It assumes that this relationship is linear in nature.

In simple terms, linear regression helps us predict a numerical value based on input features. For example, predicting house prices based on area, number of rooms, etc.

Mathematically, linear regression is represented as:

Mathematically, linear regression is represented as:

$$Y = mX + c$$

Where:

- Y = Dependent variable (output)
- X = Independent variable (input)
- m = Slope (coefficient)
- c = Intercept

In case of multiple variables, it is written as:

$$Y = b_0 + b_1X_1 + b_2X_2 + \dots + b_nX_n$$

Working of Linear Regression:

1. It takes input features (independent variables).
2. It tries to fit a straight line (best-fit line) through the data points.
3. The model calculates coefficients (weights) to minimize the error.
4. The error is usually calculated using Mean Squared Error (MSE).

Types of Linear Regression:

1. **Simple Linear Regression** – One input variable
2. **Multiple Linear Regression** – More than one input variable

Applications:

- House price prediction
- Sales forecasting
- Risk assessment
- Stock market analysis

Q2. What is a Deep Neural Network?

Ans:

A Deep Neural Network (DNN) is an advanced machine learning model inspired by the structure and working of the human brain. It consists of multiple layers of artificial neurons that process data and learn patterns to make predictions or classifications.

A neural network is called “deep” when it has more than one hidden layer between the input and output layers.

Structure of Deep Neural Network:

A Deep Neural Network mainly consists of three types of layers:

1. **Input Layer:**
 - This layer receives the input data (features).
 - Example: In house price prediction, inputs like number of rooms, area, etc.
2. **Hidden Layers:**
 - These are intermediate layers where actual processing happens.
 - Each neuron applies weights, bias, and activation functions.
 - Multiple hidden layers help in learning complex patterns.
3. **Output Layer:**
 - Produces the final result.
 - Example: Predicted house price.

Working of Deep Neural Network:

1. Input data is passed through the network.
2. Each neuron performs calculations using weights and bias.
3. Output is generated and compared with actual value.
4. Error is calculated.
5. Using **backpropagation**, weights are adjusted to reduce error.
6. This process repeats until the model achieves good accuracy.

Features of Deep Neural Network:

- Can handle large and complex datasets
- Automatically extracts important features
- Learns non-linear relationships
- High accuracy compared to traditional methods

Applications:

- Image and speech recognition
- Natural Language Processing (NLP)
- Recommendation systems
- Medical diagnosis
- House price prediction

Q3. What is the concept of Standardization?

Ans:

Standardization is a data preprocessing technique used in machine learning to transform input features so that they have a mean of 0 and a standard deviation of 1. It is also known as feature scaling.

It ensures that all input features are on the same scale, which helps the model learn more efficiently.

Mathematical Formula:

Standardization is calculated using the formula:

$$Z = \frac{X - \mu}{\sigma}$$

Where:

- **X** = Original value
- **μ (mu)** = Mean of the feature
- **σ (sigma)** = Standard deviation
- **Z** = Standardized value

Why Standardization is Important:

1. **Improves Model Performance:**
Features with large values do not dominate smaller ones.
2. **Faster Convergence:**
Helps optimization algorithms (like gradient descent) work faster.
3. **Better Accuracy:**
Ensures balanced contribution of all features.
4. **Required for Neural Networks:**
Deep Neural Networks perform better when inputs are standardized.

Working of Standardization:

1. Calculate the mean of each feature.
2. Calculate the standard deviation.
3. Subtract the mean from each value.
4. Divide by the standard deviation.

After this transformation:

- Mean becomes 0
- Standard deviation becomes 1

Example:

If a feature like house area has large values and another feature like number of rooms has small values, standardization brings both to the same scale.

Q4. Why split data into train and test?

Ans:

In machine learning, the dataset is divided into two parts: **training data** and **testing data**. This process is known as data splitting.

It is done to evaluate how well a model performs on unseen data.

Types of Data Split:

1. **Training Data:**
 - Used to train the model.
 - The model learns patterns and relationships from this data.
2. **Testing Data:**
 - Used to evaluate the model.
 - It checks how well the model performs on new, unseen data.

Why Data Splitting is Important:

1. **Avoid Overfitting:**

If we test on the same data used for training, the model may memorize instead of learning. Splitting helps detect overfitting.
2. **Evaluate Model Performance:**

Testing data gives a realistic estimate of model accuracy.
3. **Generalization:**

Ensures that the model works well on new data, not just the training dataset.
4. **Model Validation:**

Helps in comparing different models and choosing the best one.

Common Split Ratios:

- 70% Training and 30% Testing
- 80% Training and 20% Testing

Working Process:

1. Dataset is divided into training and testing sets.
2. Model is trained using training data.
3. Predictions are made on testing data.
4. Performance is measured using metrics like accuracy, MSE, etc.

Q5. Write down applications of Deep Neural Network.

Ans:

Deep Neural Networks (DNNs) are widely used in various fields due to their ability to learn complex patterns from large datasets. They are powerful tools in modern Artificial Intelligence and Machine Learning applications.

Applications of Deep Neural Network:

1. **Image Recognition:**
DNNs are used to identify objects, faces, and patterns in images.
Example: Face detection in mobile phones.
2. **Speech Recognition:**
Used to convert spoken language into text.
Example: Voice assistants like Google Assistant, Siri.
3. **Natural Language Processing (NLP):**
Helps machines understand and process human language.
Example: Chatbots, language translation.
4. **Healthcare:**
Used for disease detection, medical image analysis, and diagnosis.
Example: Detecting tumors in MRI scans.
5. **Recommendation Systems:**
Suggest products, movies, or content based on user preferences.
Example: Netflix, Amazon recommendations.
6. **Finance:**
Used for fraud detection, stock price prediction, and risk analysis.
7. **Autonomous Vehicles:**
Helps in object detection, lane detection, and decision-making in self-driving cars.
8. **House Price Prediction:**
Used to predict property prices based on multiple features like area, location, etc.

Advantages:

- Handles large and complex data
- Learns automatically without manual feature extraction
- High accuracy in predictions

Q1. What is Multiclass Classification?

Ans:

Multiclass Classification is a type of classification problem in machine learning where the output variable can belong to more than two classes. In this method, the model classifies input data into one of multiple possible categories.

Unlike binary classification (which has only two classes), multiclass classification deals with three or more classes.

Multiclass classification is a supervised learning technique where a model is trained to assign input data into one of several predefined classes.

Example:

- Recognizing alphabets (A to Z) in OCR system
- Classifying types of fruits (apple, banana, mango)
- Digit recognition (0–9)

In the given assignment, the **OCR letter recognition dataset** classifies input into **26 classes (A–Z)**.

Working of Multiclass Classification:

1. Input data with multiple class labels is provided.
2. The model learns patterns from training data.
3. It assigns probabilities to each class.
4. The class with the highest probability is selected as output.

Techniques Used:

- Softmax function (commonly used in neural networks)
- One-vs-All (OvA) method
- One-vs-One (OvO) method

Applications:

- Optical Character Recognition (OCR)
- Image classification
- Speech recognition
- Medical diagnosis (multiple disease categories)

Q2. Explain Binary Classification with an example.

Ans:

Binary Classification is a type of supervised machine learning technique in which the output variable has only two possible classes or categories. The model predicts whether a given input belongs to one class or the other.

Binary classification is the process of classifying data into two distinct classes, usually represented as 0 and 1, True/False, or Yes/No.

Example:

A common example is **movie review classification**:

- **Positive Review** → **Class 1**
- **Negative Review** → **Class 0**

In the assignment, the IMDB dataset is used to classify movie reviews as positive or negative based on text content.

Working of Binary Classification:

1. The model is trained using labeled data with two classes.
2. Input features are processed by the model.
3. The model calculates the probability of belonging to a class.
4. Based on a threshold (usually 0.5), the output is classified into one of the two classes.

Algorithms Used:

- Logistic Regression
- Support Vector Machine (SVM)
- Decision Trees
- Deep Neural Networks

Performance Evaluation Metrics:

- **Accuracy:** Correct predictions out of total predictions
- **Precision:** Correct positive predictions
- **Recall:** Ability to find all positive cases
- **F1-Score:** Balance between precision and recall

(These metrics are also mentioned in performance analysis of the assignment.)

Applications:

- Spam email detection (Spam / Not Spam)
- Disease detection (Positive / Negative)
- Fraud detection
- Sentiment analysis

1. What is Activation Function?

Simple idea:

A neural network is just doing math:

output = weight × input + bias

But this alone is **too simple (linear)**.

👉 So we use **activation functions** to make the model learn **complex patterns**.

2. ReLU (Very Important)

Formula:

$\text{ReLU}(x) = \max(0, x)$

Meaning:

- If input is positive → keep it
 - If input is negative → make it 0
-

Example:

- $\text{ReLU}(5) = 5$
 - $\text{ReLU}(-3) = 0$
-

Why we use ReLU?

- Makes model **non-linear**
 - Very fast
 - Helps deep networks learn better
-

Where used?

👉 Hidden layers

What to say in viva:

“ReLU is used to introduce non-linearity and improve learning efficiency.”

3. Sigmoid (Binary Output)

Formula:

Output between 0 and 1

Meaning:

- Gives probability
-

Example:

- 0.9 → positive
 - 0.2 → negative
-

Why used?

👉 For **binary classification**

What to say:

“Sigmoid is used when output has two classes.”

4. Softmax (Multiclass Output)

Meaning:

- Converts outputs into probabilities
 - Sum of all outputs = 1
-

Example:

[A: 0.1, B: 0.7, C: 0.2]

👉 Highest = predicted class

Why used?

👉 For multiple classes (A–Z)

What to say:

“Softmax gives probability distribution over multiple classes.”

5. What is Loss Function?

Simple idea:

Loss function = **how wrong your model is**

👉 Model tries to **minimize loss**

6. Binary Cross Entropy

Used for:

👉 Binary classification (IMDB)

Idea:

- Compares predicted probability vs actual label
-

Example:

- Actual = 1
 - Predicted = 0.9 → good
 - Predicted = 0.2 → bad
-

What to say:

“Binary cross entropy measures error for binary classification problems.”

7. Categorical Cross Entropy

Used for:

👉 Multiclass classification (OCR)

Idea:

- Compares predicted class probabilities with actual class
-

Example:

Actual = B

Prediction = [A:0.1, B:0.8, C:0.1] → good

What to say:

“Categorical cross entropy measures error for multiclass classification.”

8. Why Different Loss Functions?

Problem Type	Activation	Loss
Binary	Sigmoid	Binary Cross Entropy
Multiclass	Softmax	Categorical Cross Entropy

9. Easy Analogy (Remember This)

Think like exam:

- ReLU → “decides what to pass forward”
 - Sigmoid → “yes or no decision”
 - Softmax → “choose one from many”
 - Loss → “how wrong you are”
-

10. One-Line Summary (VERY IMPORTANT)

“ReLU is used in hidden layers for non-linearity, sigmoid is used for binary output, softmax for multiclass output, and cross-entropy loss functions measure prediction error.”

If Examiner Asks (Common Questions)

? Why ReLU?

Fast and avoids vanishing gradient problem

? Why softmax?

Because we have multiple classes

? Why sigmoid?

Because output is binary

? What is loss?

Measure of prediction error

Evaluation Parameters of Classification

When we build a classification model, we need to measure **how well it performs**. The most common metrics are based on the **Confusion Matrix**.

◆ 1. Confusion Matrix (MOST IMPORTANT)

A confusion matrix shows **actual vs predicted values**.

📌 Structure:

	Predicted Positive	Predicted Negative
Actual Positive	TP (True Positive)	FN (False Negative)
Actual Negative	FP (False Positive)	TN (True Negative)

🧠 Meaning of terms

- **TP (True Positive)** → Correctly predicted positive
- **TN (True Negative)** → Correctly predicted negative
- **FP (False Positive)** → Wrongly predicted positive
- **FN (False Negative)** → Wrongly predicted negative

Example

Suppose we predict disease:

Actual	Predicted
Sick	Sick → TP
Sick	Healthy → FN
Healthy	Sick → FP
Healthy	Healthy → TN

2. Accuracy

Formula:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

Meaning:

- Percentage of **correct predictions**

Example:

If 90 out of 100 predictions are correct → Accuracy = 90%

3. Precision

Formula:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Meaning:

- Out of predicted positives, how many are actually correct

Important when:

- False positives are costly (e.g., spam detection)

4. Recall (Sensitivity)

Formula:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Meaning:

- Out of actual positives, how many were correctly detected

Important when:

- Missing a positive is dangerous (e.g., disease detection)
-

5. F1 Score

Formula:

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Meaning:

- Balance between precision and recall
-

Q1. What is a Convolutional Neural Network (CNN)?

Ans:

A Convolutional Neural Network (CNN) is a type of deep learning algorithm that is mainly used for processing and analyzing image data. It is designed to automatically detect important features such as edges, textures, and patterns from images.

CNNs are widely used in image classification, object detection, and computer vision tasks.

Definition:

A CNN is a deep neural network that uses convolution operations to extract features from input data and classify it into different categories.

Structure of CNN:

According to the diagram on **page 2 (Figure A: High Level CNN Architecture)**, a CNN consists of three main parts:

1. **Input Layer:**
 - Takes input image (width × height × depth).
 - Example: RGB image has 3 channels.
2. **Feature Extraction Layers:**

These layers extract important features from the image:

 - **Convolution Layer:**

Applies filters (kernels) to detect features like edges.
 - **Pooling Layer:**

Reduces the size of data and helps in reducing computation.
3. **Classification Layer:**
 - Fully connected layers that classify the image into categories.

Working of CNN:

1. Input image is fed into the network.
2. Convolution layers apply filters to extract features.
3. Pooling layers reduce dimensions.
4. Extracted features are passed to fully connected layers.
5. Final output gives classification result.

Convolution Operation:

As shown in **Figure C on page 3**, convolution is a mathematical operation that combines input data with a filter (kernel) to produce a feature map.

- It acts as a **feature detector**
- Helps in identifying edges, shapes, patterns

Applications:

- Image recognition
- Face detection
- Medical image analysis
- Plant disease detection
- Fashion classification

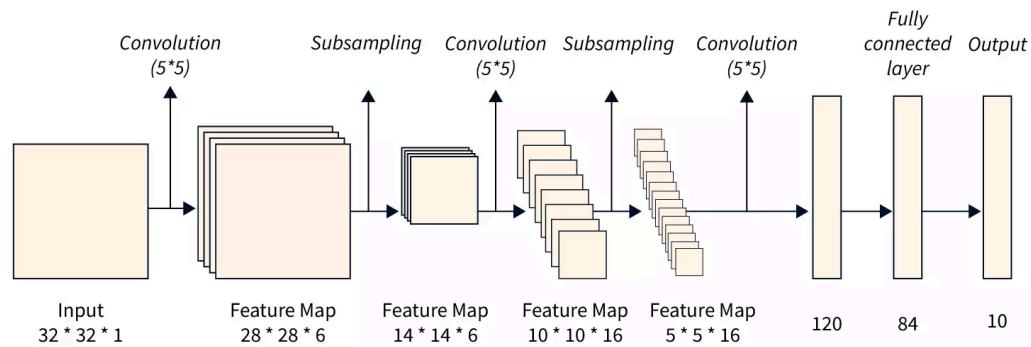
Q2. Explain LeNet-5 and AlexNet.

Ans:

1. LeNet-5:

LeNet-5 is one of the earliest and most popular Convolutional Neural Network (CNN) architectures. It was developed by **Yann LeCun** for handwritten digit recognition.

Architecture of LeNet-5:



SCALER
Topics

LeNet-5 consists of the following layers:

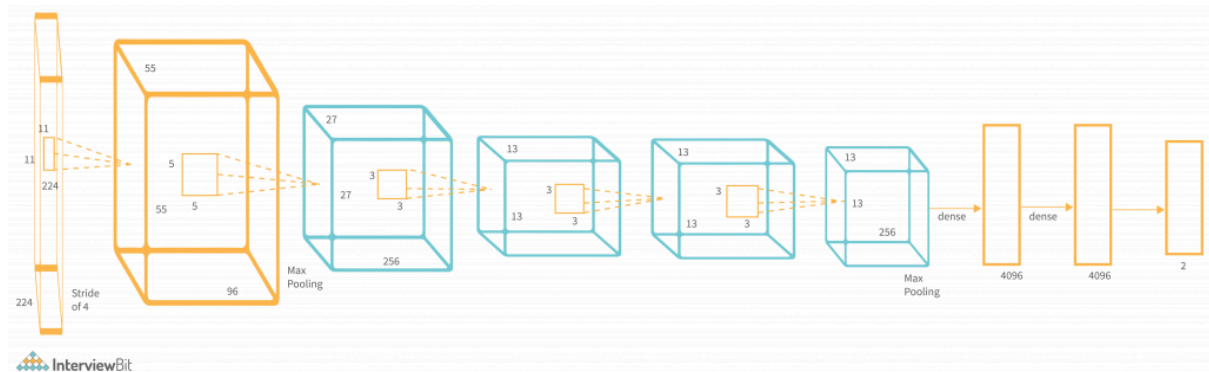
1. **Input Layer:**
 - Takes a grayscale image (32x32)
2. **Convolution Layer (C1):**
 - Extracts basic features like edges
3. **Pooling Layer (S2):**
 - Reduces size using subsampling
4. **Convolution Layer (C3):**
 - Extracts more complex features
5. **Pooling Layer (S4):**
 - Further reduces dimensions
6. **Fully Connected Layers (F5, F6):**
 - Perform classification
7. **Output Layer:**
 - Gives final output (digits 0–9)

Features of LeNet-5:

- Simple and efficient architecture
- Works well for small datasets
- Used for handwritten digit recognition (MNIST)

2. AlexNet:

AlexNet is a deep CNN architecture developed by **Alex Krizhevsky** in 2012. It became famous after winning the ImageNet competition.



Architecture of AlexNet:

AlexNet consists of:

1. **Input Layer:**
 - Takes large RGB images (224×224×3)
2. **Convolution Layers:**
 - Multiple convolution layers for feature extraction
3. **ReLU Activation Function:**
 - Introduces non-linearity and speeds up training
4. **Pooling Layers:**
 - Reduce size and computation
5. **Fully Connected Layers:**
 - Perform classification
6. **Output Layer:**
 - Uses Softmax for classification

Features of AlexNet:

- Deeper than LeNet-5
- Uses ReLU instead of sigmoid
- Uses dropout to reduce overfitting
- High accuracy on large datasets

Difference between LeNet-5 and AlexNet:

Feature	LeNet-5	AlexNet
Year	1998	2012
Depth	Shallow	Deep

Input	Small images	Large images
Activation	Sigmoid/Tanh	ReLU
Dataset	MNIST	ImageNet

Q1. What are some variants of Recurrent Neural Network architecture?

Ans:

Recurrent Neural Networks (RNNs) are designed to process sequential data. Depending on the type of input-output relationship, RNN architectures are classified into different variants.

According to the handout, these variants are based on how input sequences are mapped to output sequences.

Types of RNN Architectures:

1. One-to-One Architecture:

Input → Output
X → Y

- One input → One output
- Similar to traditional neural networks

Example: Image classification

2. One-to-Many Architecture:

→ Y1

Input → → Y2

→ Y3

- One input → Multiple outputs
- Generates a sequence from a single input

Example:

- Image captioning (image → sentence)
 - Music generation
-

3. Many-to-One Architecture:

X1 →

X2 → → Output (Y)

X3 →

- Multiple inputs → Single output
- Processes a sequence and produces one result

Example:

- Sentiment analysis (sentence → positive/negative)
-

4. Many-to-Many Architecture:

X1 → → Y1

X2 → → Y2

X3 → → Y3

- Multiple inputs → Multiple outputs
- Input and output are sequences

Example:

- Language translation
- Speech recognition

Q2. What are the limitations of Recurrent Neural Networks (RNNs)?

Ans:

Recurrent Neural Networks (RNNs) are powerful for handling sequential data, but they have several limitations which affect their performance in real-world applications.

Limitations of RNN:

1. Vanishing Gradient Problem:

- During training, gradients become very small.
 - This makes it difficult to update weights.
 - As a result, the network fails to learn long-term dependencies.
-

2. Exploding Gradient Problem:

- Gradients become very large during backpropagation.
 - This leads to unstable training and incorrect results.
-

3. Difficulty in Learning Long-Term Dependencies:

- RNNs struggle to remember information from earlier time steps.
 - They mainly focus on recent inputs.
-

4. Slow Training Process:

- RNN processes data sequentially (step-by-step).
 - Cannot be fully parallelized like CNNs.
 - Hence, training takes more time.
-

5. High Computational Cost:

- Requires more resources due to repeated calculations at each time step.
-

6. Short-Term Memory Limitation:

- As mentioned in the handout, RNN has only **short-term memory**, which limits its performance in long sequences.

7. Complex Training (BPTT):

- Uses **Backpropagation Through Time (BPTT)**
 - This process is computationally expensive and complex.
-

8. Replaced by Advanced Models:

- Modern models like **Transformers and LLMs** perform better.
- They handle long sequences more efficiently.